

Software Engineering Operations

ACRONYMS

API	Application Programming Interface
ATDD	Acceptance Test Driven Development
CD	Continuous Delivery
CI	Continuous Integration
CPU	Central Processing Unit
CONOPS	Concepts of Operations
DBMS	Database Management System
IaC	Infrastructure as-Code
IaaS	Infrastructure as a Service
IT	Information technology
ITIL	IT Infrastructure Library
KPI	Key Performance Indicator
MR	Modification request
MVP	Minimum Viable Product
PaaS	Platform as a Service
PR	Problem Report
QA	Quality Assurance
SaaS	Software as a Service
SLA	Service-Level Agreement
SRE	Site Reliability Engineering
TDD	Test Driven Development

INTRODUCTION

Software engineering operations refers to the set of activities and tasks necessary to deploy, operate and support a software application or system while preserving its integrity and stability. These activities include the deployment and configuration of the software in the

targeted operational environments and the monitoring and management of the application while it is in use (until it is retired). Once the application is operational, software engineering operations must manage any defects that are uncovered, any changes made to the system software environment and hardware equipment over time, and any new user requirements that surface.

Software engineering operations is an integral part of system and software life cycle processes [3]. The Software Engineering Operations Knowledge Area (KA) is related to all other aspects of software engineering.

Specialized software and information technology (IT) operations engineers have traditionally provided and managed IT operations services. Best practices in software engineering operations were initially published by the IT infrastructure library (ITIL) and were quickly accepted by the industry. These practices were summarized and published in the IEEE 20000 standard [1].

Historically, operations and computing centers were often located in organizational silos separate from software development activities. Progressive organizations now co-locate software development, software maintenance and some software engineering operations activities (often provided as a service and often coined DevOps). Benefits of this approach are the elimination of the organizational silos that separated these software activities and the sharing of common processes and tools. The rising popularity and growing acceptance of DevOps practices [2*] and related standards [4], including an ever-evolving set of tools, reflect this trend. DevOps aims at automating and continuously evolving software engineering activities

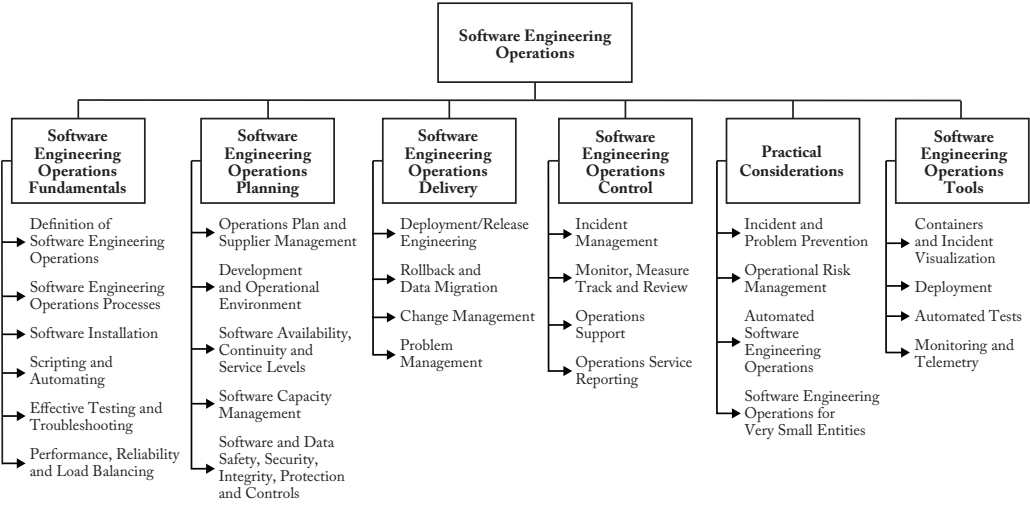


Figure 6.1. Breakdown of Topics for the Software Engineering Operations KA.

to ensure high-quality software and to satisfy users who demand quicker turnaround from software engineers.

In this context, the role of software engineers involved in software engineering operations has significantly evolved from 2015 to 2025 with the emergence of practices like Infrastructure-as-Code (IaC), Platform-as-Code (PaC), Agile infrastructure, software-defined architectures/systems, and the availability of infrastructure as a service (IaaS) and platform as a service (PaaS) solutions. Tasks traditionally performed by IT infrastructure engineers are increasingly automated and made available as a service, enabling application developers to perform software engineering operations tasks independently as part of their daily project activities. For example, application developers in many organizations can now directly use IaaS and PaaS to deploy applications in production environments and to monitor different aspects of those applications without directly involving operations engineers.

Having end-to-end resources and desired state configuration managed like code, using practices such as IaC and PaC, provides value in the form of improved repeatability,

consistency/standardization, known security policies, self-documentation (transparency), single source of truth, configuration control, and scalability. From an engineering perspective, the important point is that nearly anything that impacts a software product directly or indirectly should be considered for representation as code.

To perform software engineering operations tasks, some organizations use the concept of Platform Engineering and Site Reliability Engineering (SRE) [6] to increase productivity and software quality. The role of platform engineering is to build and manage self-service platform capabilities that can be used by software engineers to develop, deploy, and operate software applications. On the other hand, the role of SRE is to monitor, automate, and improve software operations with respect to non-functional aspects, including availability, performance, latency, and security. SRE is also responsible for change management, emergency response, capacity planning, and overall efficiency of software systems.

Although many organizations still use conventional IT operations management processes, this KA focuses mainly on the role

of software engineers in operations in the emerging contexts of DevOps, IaC, PaC, and Agile infrastructure practices.

In this context, we identify two main software engineering roles related to operations: *Operations engineer*, who is responsible for developing operations services made available as a service and accessible through an application programming interface (API), and *software engineer*, who can use the resulting operations services (available as a service) to independently deploy and manage applications without directly involving IT operations specialists.

BREAKDOWN OF TOPICS FOR SOFTWARE ENGINEERING OPERATIONS

The breakdown of topics for the Software Engineering Operations KA is shown in Figure 6.1.

1. Software Engineering Operations Fundamentals

This first section introduces the concepts and terminology that form an underlying basis for understanding the role and scope of software engineering operations.

1.1. Definition of Software Engineering Operations [1, c3s3.3][3, c6s6.4.12]

In this *Guide*, the term *software engineering operations* refers to the knowledge, skills, processes and tools used by software engineers or their organization to ensure that a software product, including IT infrastructure, system software, and application software, operates well during development, maintenance and in real conditions of operations.

In ISO/IEC/IEEE 12207 [3], an *operator* is defined as an “individual or organization that performs the operations of a system.” The *SWEBOK Guide* modifies that definition for the term *operations engineer*, which refers to a software engineer who executes software

engineering operations processes. In this role, an *operations engineer* works closely with *software engineers* to develop and offer operations services such as the following:

- Provisioning, deployment, configuration, and support for containers and virtual servers,
- Designing and offering on-demand services (e.g., environment on demand, versioning, continuous integration (CI) and testing, deployment, and surveillance) for use by software engineering,
- Monitoring and troubleshooting system and application software incidents by running diagnostics, documenting problems and resolutions, prioritizing problems, and assessing impact of issues,
- Performing, automating and implementing appropriate processes for security, data protection and failover procedures,
- Overseeing capacity, storage planning and database management system (DBMS) performance,
- Providing documentation and technical specifications to IT staff for planning and implementing new or upgraded IT infrastructure and system software.

ISO/IEC/IEEE 20000-1 describes the need to develop and enhance the professional competencies of operations engineers. To achieve this goal, software organizations should address the following:

- Staff recruitment: To validate job applicants’ qualifications and competencies, including their professional certifications, and to identify their strengths, weaknesses and potential capabilities against the operations engineer job description, core technologies and computer languages mastered and overall experience,
- Resource planning: To staff new or expanded engineering operations services, plan the use of new technology, plan the assignment of service management staff to development project teams,

- Operations Plan and Supplier Management
- Development and Operational Environment
- Software Availability, Continuity and Service Levels
- Software Capacity Management
- Software Backup, Disaster Recovery and Failover
- Software and Data Safety, Security, Integrity, Protection and Controls

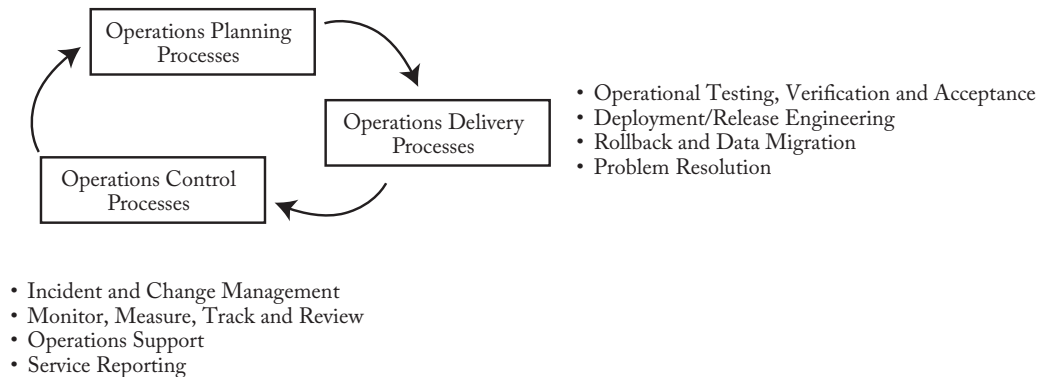


Figure 6.2. Software Engineering Operations Processes and Activities

develop succession planning and other staffing gaps created by staff turnover,

- Resource training and development: To identify training and development requirements and create a training and development plan that meets them; also, to provide timely, effective delivery of operations services. Operations engineers should be trained in the relevant aspects of service management (e.g., via training courses, self-study, mentoring and on-the-job training), and their teamwork and leadership skills should be developed. A chronological training record should be maintained for each individual, with descriptions of the training provided.

1.2. Software Engineering Operations Processes [2*, s1][3, c6s6.4.12]

ISO/IEC/IEEE 20000-1 is the reference standard that presents an overview of operations processes. It specifies requirements for the design, transition, delivery and improvement of operations services. The ISO/IEC/IEEE 20000-1 describes five main operations process groups:

service delivery processes, release processes, control processes, resolution processes and relationship processes. These operations processes are further categorized as technical processes in ISO/IEC/IEEE 12207 [3]. Operations processes, from the perspective of a software engineer, contain the activities and tasks necessary to deploy, configure, operate and support an existing software system or product while preserving its integrity. This international standard describes four main operations process activities: 1) prepare for the operation: that requires to define an operation strategy; 2) perform the operation: which consist of operating and monitoring; 3) manage the results of operation: where anomalies are recorded and addressed; and finally 4) support the customer: which means to give assistance and consultation to any user of the operations services.

Finally, ISO/IEC/IEEE 32675 [4] introduces a number of software engineering operations activities using an Agile and a minimum viable product (MVP) perspective. This standard recognizes the influence of DevOps as a set of principles and practices that enable better communication and collaboration between

relevant stakeholders for the purpose of specifying, developing, continuously improving, and operating software and system products and services. These processes and activities are the responsibility of operations engineers.

For the purpose of the *SWEBOOK Guide*, engineering operations activities can be grouped into three main operations processes (see Figure 6.2) that each contain a number of operations activities, which are described in the following sections of this chapter:

- Operations Planning (section 2),
- Operations Delivery (section 3),
- Operations Control (section 4).

Each software engineering operations process includes activities performed during the pre delivery and post delivery stages of a software project. Software engineering operations planning activities occur during the pre delivery stage. These activities are covered in this chapter.

1.3. *Software Installation*

[1, c3, c6s2][2*, c3s3.1]

Before a software application or update can be made available to the users (i.e. released in production), the operations engineer must install the software as part of its deployment. To install the software, the engineer might have to uninstall previous versions, configure the software for its target destination, and create the necessary directories, registry files and environment variables on the target destination. This is often done using a scripting language. The installation of the software to the appropriate locations is typically done electronically, but in the case of embedded systems, it might require the use of a physical medium. Once the software is installed, a verification step is conducted to ensure that the operation succeeded.

1.4. *Scripting and Automating*

[2*, c9]

As part of software engineering operations, repetitive tasks are automated to reduce

delays, increase quality, and ensure a consistent and stable operational environment. This is typically achieved using scripting languages, which are basic programming languages. Automating operations enables a quicker reaction in case of a failure and, therefore, results in less downtime and fewer severe incidents, as alerts are sent immediately. Automating such tasks is also a good way to ensure standardization of operations in an organization. It also constitutes the basis for the development of operations made available as a service. Refer to section 6 for further discussion on operations tools.

1.5. *Effective Testing and Troubleshooting*

[2*, c3]

Software engineering operations is responsible for ensuring the stability of the system. For this purpose, software must be thoroughly tested before it is released (deployed in production and made available to users). Because manual testing is inefficient, error-prone and non-scalable, testing must be automated as much as possible throughout the entire software process. Also, because the time available for testing is limited, regression testing and test coverage strategies (the selective retesting of a software application, or component, to verify that the software to be deployed will not cause unintended effects) play an important role in software engineering operations.

When errors are found (in production after the software is released or during internal testing phases), software engineers and software operations engineers need to troubleshoot hardware and software incidents by running diagnostics, documenting problems and resolutions, prioritizing problems, and assessing the impact of the issues. The cost — in both time and money — of repeating full testing on a major piece of software is significant. To ensure that the requested problem reports (PRs) are valid, the operations engineer should replicate and verify problems by running the appropriate tests. Testing certain aspects of the software in production can be particularly

challenging. For example, when software performs critical functions, bringing it off-line to test might be difficult. Generally, testing the software in the production system context is challenging (sometimes impossible) and could require the use of testing techniques such as canary testing and dark launches. The Software Testing KA provides additional information and references on testing.

1.6. *Performance, Reliability and Load Balancing* [1, c6s6.2]

Software operations engineers plan for performance, reliability and load balancing early in software projects to ensure they meet the project requirements. (See section 1.2 to 1.7 of the Software Requirements KA). A current trend is for software engineers to design and use infrastructure/operations services to adjust dynamically (e.g. scalability) the infrastructure according to the demand. Using DevOps practices enables operations engineers to anticipate these needs early and provide infrastructure services that software engineers can use and test during the development stages of a project.

2. **Software Engineering Operations Planning**

This topic introduces some of the generally accepted techniques used in software engineering operations planning. Operations engineers must deal with a number of key issues to ensure software operates effectively. Operations engineers should document their software engineering operations steps and tools, using any type, form or medium suitable for the purpose (e.g., Wikis, documents, and more). The following topics are typically considered suitable as evidence of well documented operations:

- Policies and plans,
- Service documentation,
- Procedures,
- Processes, and
- Process control records.

2.1. *Operations Plan and Supplier Management* [1, c4s4.1][3, c6s6.1]

Software engineering operations planning should comprise part of the process of translating project requirements and the needs of the developers and maintainers into services, and it should provide a road map for directing progress. This process often involves the products and services of suppliers that must be well coordinated to ensure quality service. ISO/IEC/IEEE 20000-1 describes planning activities, as well as ISO/IEC/IEEE 12207, which lists the activities operations engineers considers from human, technical and system perspectives.

2.1.1. *Operations Plan* [1, c4s4.1][3, c6s6.4.12.3a]

Whereas software development typically lasts from some months to a few years, the operations phase usually lasts many years. Therefore, estimating resources is a key element of operations planning. Software engineering operations planning should begin with the decision to develop a new software product and should consider its maintenance and operations requirements early. A concept document should be developed, followed by an operations and maintenance plan [1, c7s2], and both should address the following:

- Scope of the operations and software maintenance,
- Adaptation of the software engineering operations process and tools,
- Identification of the software engineering operations organization,
- Estimate of software engineering operations and maintenance costs.

The next planning step suggests to develop a software engineering operations plan, or concept of operations (CONOPS). This plan should be prepared during software development and should specify how users will request software modifications and report problems or issues when the software will be operational.

Software engineering operations planning is addressed in ISO/IEC/IEEE 12207 [3] and ISO/IEC/IEEE 32675 [4]. The standards provide guidelines for planning, implementing, maintaining, automating and supporting production software. Finally, at the highest planning level, the operations organization must conduct business planning activities (e.g., budgetary, financial and human resources), just as all the other divisions of the organization (refer to the Software Engineering Management KA). ISO/IEC/IEEE 20000-1 recommends that the operations plan addresses issues associated with a number of planning perspectives, including the following:

- The roles and responsibilities for implementing, operating and maintaining the new or changed service,
- Activities to be performed by customers and suppliers,
- Changes to the existing service management framework and services,
- Communication to the relevant parties,
- New or changed contracts and agreements to align with changes in business needs,
- Staffing and recruitment requirements,
- Skills and training requirements (e.g., users, technical support),
- Processes, measures, methods and tools to be used in connection with the new or changed service,
- Capacity management,
- Financial management,
- Budgets and timescales,
- Service acceptance criteria, and
- The expected outcomes from operating the new service, expressed in measurable terms.

This plan ensures that an operational strategy is defined, conditions for correct operations are identified and evaluated, the software is tested at scale to operate in its intended environment, and surveillance is provided to ensure responsiveness and availability of the software by ensuring constant support. At the individual request level (e.g., problem report (PR) or modification request

(MR)) planning is required. Once individual requests are received and validated, the release or version planning activity requires that operations engineers perform the following tasks:

- Identify the target availability dates of individual requests,
- Agree on the content of subsequent releases or versions,
- Identify potential conflicts and develop alternatives,
- Assess the risk of a given release and develop a rollback and data migration plan (see section 3.3) in case problems arise,
- Inform all stakeholders.

2.1.2. *Supplier Management*

[1, c7s3][3, c6s6.1]

Supplier management ensures that the organization's suppliers and their performance are managed appropriately to support the seamless provision of quality products and services. ISO/IEC/IEEE 12207 lists the activities that the operations engineer will perform to establish an agreement to acquire suppliers' products and/or services. From an operations engineer's perspective, the nature of the relationship with suppliers and the approach should be determined by the nature of the products and services needed in a project. Managing suppliers of services related to operational software includes managing out-sourced services and cloud services, like IaaS and PaaS.

2.2. *Development and Operational Environments*

[2*, c9]

The overall software process requires the use of different environments at different stages. These are typically defined as the development environment, the testing or quality assurance (QA) environment, the preproduction environment, and the production environment. To build quality into the product and reduce the risks associated with the release of software in the production environment (whether the release is associated with new functionality or software defects), engineers must

ensure that the different environments are all coherent and synchronized with the production environment.

For this reason, DevOps recommends that the creation of all the different environments be automated and built from a single code repository. In mature DevOps organizations, the creation of the different environments is completely automated and made available as a service. Also, all environments need to be built from the same code source (single source of truth) to ensure that all the environments are synchronized with the production environment in which the software is released. This leads to the concept of infrastructure as code (IaC).

2.3. *Software Availability, Continuity, and Service Levels* [1, c6s6.3]

Service availability and continuity must be managed to ensure that customer commitments are met. Because service availability and continuity are defined as nonfunctional requirements early in a project (see the Software Quality KA), operations engineers will ensure that the proper infrastructure is planned, designed, implemented and tested. Software availability is measured and recorded, and unplanned nonavailability is investigated and appropriate actions taken. Service reports produce availability and continuity indicators of operations services against service-level targets.

The service-level management process monitors the agreed software level of service, including workload characteristics, performance and availability trend information and customer satisfaction analysis. Defining, agreeing to and documenting service-level agreements (SLAs) can help clarify the full range of operations services obligations provided. The Software Maintenance KA provides additional information and references about SLAs.

2.4. *Software Capacity Management* [1, c6s6.5]

ISO/IEC/IEEE 20000-1 describes the need to ensure that the software product has the

capacity, at all times, to meet current and future agreed-upon demands created by the customer's business needs. The current and expected business requirements for services should be understood in terms of what the business needs in order to deliver its products or services to its customers. Business predictions and workload estimates should be translated into specific requirements and documented. The reaction to variations in workload or environment should be predictable; data on current and previous components, as well as resource utilization at an appropriate level, should be captured and analyzed to support the process.

Capacity management is the focal point for all performance and capacity issues. The process should directly support the development of new and changed services by sizing and modeling these services. A capacity plan documenting the actual performance of the infrastructure and the expected requirements should be produced at a suitable frequency (at least annually), considering the rate of change in services and service volumes, information in the change management reports, and changing customer business requirements. The capacity plan should document costed options for meeting business requirements and recommend solutions to ensure achievement of the agreed-upon service-level targets as defined in the SLA. The technical infrastructure and its current and projected capacities should be well understood to ensure optimal software operations.

2.5. *Software Backup, Disaster Recovery, and Failover* [1, c6s6.3.4]

ISO/IEC/IEEE 20000-1 also proposes that the following should be quickly available following a major service failure or disaster to ensure continuity planning and testing: backups of data, documents and software, and any equipment or staff necessary for service restoration. Backup and data recovery are important activities; successful recovery is especially vital. The need for successful recovery should influence which backup and

recovery methods are used (full or incremental), how frequently restore points are established, where they are stored, and how long they are retained.

Preparedness and regular test of backup, disaster recovery, and failover should be constantly rehearsed as changes to the production environment are made. This is another essential activity that is triggered when outage assessments are done. Testing disaster recovery requires stopping the service, identifying the checkpoint state and triggering the failover process. Software engineers should understand that failure is inevitable and that automated failover daemons can reduce recovery time drastically. To achieve this, software applications should include failure-handling logic; this must be planned during development. DevOps can help organizations that want to reduce failovers and disasters by automating and launching tests as often as possible to ensure readiness in case of a failure or catastrophic event.

2.6. Software and Data Safety, Security, Integrity, Protection, and Controls
[1, c6s6.6]

The need to manage information security effectively within all service activities is described in ISO/IEC/IEEE 20000-1. This is done by conducting a software risk assessment on the security and availability of information. Operations engineers should strive to enforce the following controls:

- a. Senior management should define their information security policy, communicate it to staff and customers, and act to ensure its effective implementation,
- b. Information security management roles and responsibilities should be defined and allocated to post holders,
- c. A representative of the management team should be assigned the role of monitoring and maintaining the effectiveness of the information security policy,
- d. Staff with significant security roles should receive information security training,
- e. All staff should be made aware of the information security policy,
- f. Expert help on risk assessment and control implementation should be available,
- g. Changes should not compromise the effective operation of controls, and
- h. Information security incidents should be reported following incident management procedures, and a response should be initiated.

In line with the evolution of DevOps, DevSecOps is promoting the integration of security early and throughout the software process, which includes the integration of different security mechanisms and tools at the operations level. The goal is to automate the detection and correction of security issues as early as possible in the overall process.

3. Software Engineering Operations Delivery

This topic introduces some of the generally accepted processes used during software engineering operations delivery (ISO/IEC/IEEE 20000-1): SLA, service reporting, service continuity, availability management, budgeting and accounting for IT services, capacity management, and information security management.

3.1. Operational Testing, Verification, and Acceptance [2*, c10] [3, c6s6.3.5.3d]

Software engineers plan and execute software verification as early as possible, using test-driven development (TDD) and acceptance test-driven development (ATDD) techniques and tools that ensure that operational testing is ongoing during the development of the software, not only at the end of a project. DevOps plays an important role in developing and automating software testing services and integrating different tools to improve software productivity and quality. (See TDD and ATDD in the Software Testing KA.)

3.2. Deployment/Release Engineering

[2*,c12][3,c6s6.3.5.3d]

A software operations engineer's main responsibility relates to the deployment and release of software to ensure its continued performance. As defined in [2*], "deployment is the installation of a specified version of software to a given environment (e.g., deploying code into an integration test environment or deploying code in production)," whereas "release is when we make a feature (or set of features) available to all our customers or a segment of customers (e.g., we enable the feature to be used by 5% of our customer base)." Release processes include all the activities related to release management. ISO/IEC/IEEE 12207 [3] lists release control activities and explains the need to identify and record release requests, identify the software system elements in a release followed by approval, and track the releases in their specified environments.

DevOps advocates integrating development and operations in the same team to improve software engineering operations efficiency. In traditional software processes, when an application is ready for deployment, it is transferred from a development team to an operations team that is responsible for deployment, which is mostly done manually. This results in processes that are inefficient from both a time and a quality perspective. To improve the efficiency of the deployment process, DevOps calls for automating the different deployment steps, including packaging the code, generating configuration files, restarting the servers, configuring the servers and databases, installing the software on the different servers, launching the execution of the application, and executing smoke testing.

Different release engineering strategies can be used to reduce the risks associated with software releases. These strategies can be grouped into two main categories: environment-based release strategies and application-based release strategies. Environment-based release strategies use a

staging environment to support the release of a new version of an application. In other words, the basic strategy involves deploying the new version of the application to a staging environment. Application-based release strategies are based on the use of toggles (e.g., feature toggles) that make it possible to enable or disable specific sections of the code (e.g., a feature) using configuration parameters.

Deployment and release are supported by automation techniques and tools. The canary release testing technique is a partial and time-limited deployment of a change in a service and an evaluation of that change. This evaluation helps the operations engineer decide whether to proceed with a complete deployment. Similarly, tools that manage the installation of new software typically observe the newly started software for a while, ensuring that the software doesn't crash or otherwise misbehave. The same technique is useful for observing recent changes; if they do not pass the validation period, they can be automatically rolled back. The Software Configuration Management KA provides more information about the release processes. Once the application platform is deployed in the targeted production environment, the decision to make it available to the users (release it) becomes a business decision.

3.3. Rollback and Data Migration

[2*,c12][3,c6s6.4.10.3]

Rollback and *data migration* are terms used to describe the process of returning software and its database to a state where they work properly. Software engineers ensure that when a new version of the software and its databases have been modified and deployed to production, they can easily and quickly be rolled back in case the new version is causing defects or product degradation in production. This means a planned and rehearsed rollback is done before a new version of the software is deployed in production. DevOps processes automate this process to make it faster; in fact, the automated surveillance can trigger rollback and data migration to a previous state so

quickly that the end user doesn't notice that there was a problem. Both release strategy categories (described in section 3.2) — environment-based release and application-based release — can be used to support rollback.

3.4. *Problem Resolution* [1, c8s8.3]

The objective of this operations process is to minimize disruption to the business through the identification and analysis of the cause of software and system incidents and problems. This approach may require the involvement of a multidisciplinary team, whose software engineers and operations engineers investigate, for example, recurring production problems that might have an underlying cause in software infrastructure and system components. This might require monitoring, logging and profiling the software and its infrastructure behavior.

4. **Software Engineering Operations Control**

This topic introduces some generally accepted techniques used in software engineering operations control.

4.1. *Incident Management* [1, c8s8.2]

Incident management is the process of recording, prioritizing and assessing the business impact, resolution, escalation and closure of software incidents. The modern DevOps approach automates software surveillance using alerts and logs to prevent minor incidents from becoming major incidents. When an incident occurs, proper analysis and/or post mortems must be conducted to find the source of the incident and appropriate solutions must be implemented to prevent similar incidents to happen again in the future.

4.2. *Change Management* [1, c9s9.2]

This operations process ensures that all changes are assessed, approved, implemented

and reviewed in a controlled manner. All change requests are recorded and classified (e.g., emergency, urgent, major and minor). This process assesses the risk of a change and the need for a rollback strategy in case of failure. Large systems might require that a change schedule be planned with the product manager and end users.

Whereas in traditional software delivery processes (or software life cycle models), all changes are delivered as part of new software releases (containing multiple changes related to different aspects of the application or system) issued at fixed time intervals (e.g., every three months), DevOps aims to deliver small units of change (a single new functionality or service, or defect fix, rather than a new version of an application containing multiple changes) on demand and independently from each other. For this purpose, software applications (or services) must be architected to enable small, independent software deployments.

4.3. *Monitor, Measure, Track, and Review* [2*, c14-15]

Software engineering operations activities monitor capacity, continuity and availability. In a DevOps mindset, hope should not be a strategy; instead, engineers should be informed about system quality and operational health with evidence, such as the following key performance indicators (KPI), which are available to stakeholders in real time:

- Production system's monitoring and product telemetry,
- Actionable verification and validation results before and after release to production,
- End-user activity and resource use,
- Impact analysis results,
- Inter- and intra-related dependencies required for system operation,
- Configuration changes unrelated to approved deployment tasks, and
- Security and resilience performance capability.

4.4. *Operations Support* [1, c6, c14s5]

ISO/IEC/IEEE 12207 [3], ISO/IEC/IEEE 20000-1 [1] and ISO/IEC/IEEE 32675 [4] identify the primary software engineering operations activities that support the operations processes — activities that operate the software product in its intended environment — and the primary activities that provide support to the customers of the software products. Operations support activities are initiated at the planning stage of the project and are then executed, which often requires techniques and tools to proactively monitor the product and services and react quickly to events and incidents. Support activities are often described in SLAs.

4.5. *Service Reporting* [1, c6s6.2]

Service reporting aims to produce agreed-upon, timely, reliable and accurate information for decision-making. Each service report helps demonstrate how an operations service has performed and whether it has met some stated and agreed-upon end-user objective. Typical service reports address performance against service-level targets, as well as security breaches, the volume of transactions and resource use, incidents and failures, trend information, and satisfaction analysis. Operations engineers need to establish automated systems and tools for measurement to do the following:

- Determine whether measures are already available or additional instrumentation for collection, analysis and reporting is needed,
- Select or develop a framework and tools to allow coordination of measurement collection for analysis, reporting and control.

5. **Practical Considerations**

This topic introduces practical considerations for software engineering operations.

5.1. *Incident and Problem Prevention* [2*, c7]

The overall operations process needs to be automated as much as possible to prevent incidents and problems, and automated testing needs to be integrated throughout the process. Also, product telemetry should be implemented with proper analytics techniques to detect problems as early as possible to prevent incidents. For this purpose, data collected at all layers of the product stack (including application layer, operating system layer and infrastructure layer) must be collected and analyzed. Using product telemetry not only allows engineers to detect potential issues but also provides the foundation for identifying the source of the problem.

5.2. *Operational Risk Management* [3, c6s6.4.12.3c4]

Operations engineers must manage a number of risks. IEEE 2675 [4] defines continuous risk management as a continuous process that can be automated to monitor operations constantly for risks that can affect software availability, scalability and security. Operations engineers can take measures to automate the alerts. To decide what events will trigger an alert, they need to talk with product owners and software engineers to establish an agreed-upon level of risk tolerance. Other perspectives are to choose the deployment process that is appropriate for the risk profile of a given service and the risks of exposing private data.

5.3. *Automating Software Engineering Operations* [2*, c8]

Automation is playing an important role in modern software operations. Software engineers achieve the best results when coupling applications and operations automation. Although automation primarily focuses on managing the life cycle of a system or infrastructure (e.g., user account creation, environments and server provisioning, runtime

config changes), it can also be useful in other use cases where services can be developed to help software engineers deploy, test and debug during development. Trends in operations automation aim to reduce complexity, accelerate provisioning of infrastructure, offer operations services scripts to developers, define applications, automate deployment and test workflows.

5.4. *Software Engineering Operations for Small Organizations*

Very small organizations (organizations of up to 25 people) have difficulty applying standards developed by and for large organizations, as their requirements can overwhelm the capabilities of small organizations. This is where the ISO/IEC 29110 series of standards is useful, as it provides standards and guidelines adapted to very small organizations to ensure the quality of their software engineering operations [7]. Software engineers should be aware that operations processes can be adapted to small organizations and that the ISO/IEC CD 29110-5-5 will be addressing this purpose.

6. Software Engineering Operations Tools [1, c5s5g][2*, c12]

This topic encompasses tools that are particularly important in software engineering operations for maximizing the efficient use of personnel. Automating development, maintenance and operations-related tasks saves engineering resources and improves quality and turnaround. When implemented appropriately, such automated tasks are generally faster, easier and more reliable than they would be if they were attempted manually by software engineers and operations engineers. DevOps supports such automation for integrating, building, packaging, configuring, and deploying reliable and secure systems. It combines development, maintenance, and operations resources and procedures to perform CI, delivery, testing and deployment.

Continuous delivery (CD) is a software engineering practice that uses automated tools to provide frequent releases of new systems (including software) to staging or various test environments. CD continuously assembles the latest code and configuration from the head into release candidates.

Continuous testing is a software testing practice that involves testing the software at every stage of the software development life cycle. Continuous testing aims to evaluate the quality of software at every step of the CD process by testing early and often. Continuous testing involves various stakeholders, such as developers, DevOps personnel, and QA and end-users.

Continuous deployment (aka CD) is an automated process of deploying changes to production by verifying intended features and validations to reduce risk. Jez Humble and David Farley [8] pointed out that “[t]he biggest risk to any software effort is that you end up building something that isn’t useful. The earlier and more frequently you get working software in front of real users, the quicker you get feedback to find out how valuable it really is.”

6.1. *Containers and Virtualization*

Different container/virtualization technologies and management tools (also called orchestrators) are available to operations engineers to improve the scalability of applications and standardize software deployment across multiple computer and server suppliers. [4, c6s6.4.12] Operations engineers use their knowledge of the size and complexity of each project to identify the best tool for flexibility, security and monitoring.

6.2. *Deployment* [2*, c12]

Different technologies and tools can be used to manage software deployments in different environments. [4, c5s5.1] Also, different tools are usually combined to cover the different phases and aspects of software deployment, ranging from the specification of deployment and configuration using descriptor files to the

automated deployment and management of production environment resources.

6.3. Automated Test [2*, c10]

To enable fast and constant feedback to the developers, testing must be automated as much as possible throughout the entire software delivery process, including throughout development and operations. For this purpose, a testing strategy covering the different types of test (unit test, integration test, system test, user acceptance test) must be defined, and tools to support and automate the different testing phases must be selected. The automation of testing is critical to provide continuous feedback to software engineers developing code and thereby to improve software quality.

6.4. Monitoring and Telemetry [2*, c14-15]

Monitoring and telemetry are key aspects of software engineering operations. They

collect data at all layers of the software system (including application, operating system and server) and extract information that can be used to analyze and monitor different aspects of the system to detect issues and follow the evolution of various properties. James Turnbull [9] describes a general monitoring framework architecture used by engineering operations in many technology organizations. Implementing monitoring solutions requires combining different techniques and tools to collect data at different layers. This includes logs at the application level, execution traces at the operating system level and resource use information (like CPU and memory use) at the server level. Then, based on the collected data, different analytics techniques (e.g., statistical analysis and machine learning techniques) can be used to extract relevant information. Finally, dashboards can be used to visualize the extracted information; different dashboards can be developed to display relevant information to different stakeholders.

MATRIX OF TOPICS VS. REFERENCE MATERIAL

	ISO 20000-1 [1]	The DevOps Handbook [2*]	ISO 12207 [3]
1. Software Engineering Operations Fundamentals			
1.1. Definition of Software Engineering Operations	c3s3.3		c6s6.4.12
1.2. Software Engineering Operations Processes	s1		c6s6.4.12
1.3. Software Installation	c3, c6s6.2	c3s3.1	
1.4. Scripting and Automating		c9	
1.5. Effective Testing and Troubleshooting		c3	
1.6. Performance, Reliability and Load Balancing	c6s6.2		
2. Software Engineering Operations Planning			
2.1. Operations Plan and Supplier Management	c4s4.1		c6s6.1
2.2. Development and Operational Environments		c9	

2.3. <i>Software Availability, Continuity and Service Levels</i>	c6s6.3		
2.4. <i>Software Capacity Management</i>	c6s6.5		
2.5. <i>Software Backup, Disaster Recovery and Failover</i>	c6s6.3.4		
2.6. <i>Software and Data Safety, Security, Integrity, Protection and Controls</i>	c6s6.6		
3. Software Engineering Operations Delivery			
3.1. <i>Operational Testing, Verification and Acceptance</i>		c10	c6s6.3.5.3d
3.2. <i>Deployment/Release Engineering</i>		c12	
3.3. <i>Rollback and</i>			
Data Migration			
3.4. <i>Change Management</i>	c9s9.2		
3.5. <i>Problem Management</i>	c8s8.3		
4. Software Engineering Operations Control			
4.1. <i>Incident Management</i>	c8s8.2		
4.2. <i>Monitor, Measure, Track and Review</i>		c14-15	
4.3. <i>Operations Support</i>	c6, c14s5		
4.4. <i>Operations Service Reporting</i>	c6s6.2		
5. Practical Considerations			
5.1. <i>Incident and Problem Prevention</i>		c7	
5.2. <i>Operational Risk Management</i>			c6s6.4.12.3
5.3. <i>Automating Software Engineering Operations</i>		c8	
5.4. <i>Software Engineering Operations for Small Organizations</i>			
6. Software Engineering Operations Tools	c5s5g	c12	
6.1. <i>Containers and Virtualization</i>			
6.2. <i>Deployment</i>		c12	
6.3. <i>Automated Test</i>		c10	
6.4. <i>Monitoring and Telemetry</i>		c14-15	

REFERENCES

- management — Part 1: Service management systems requirements*, ed. IEEE, 2013.
- [1] IEEE standard, *ISO/IEC/IEEE 20000-1:2013, Information technology — Service*
- [2*] G. Kim, J. Humble, J. Debois, J. Willis, and N. Forsgren, *The DevOps*

- Handbook: How to create world-class agility, reliability and security in technology organizations*, 2nd ed., IT Revolution Press, 2021.
- [3] IEEE standard, *ISO/IEC/IEEE 12207:2017, Systems and software engineering — Software Life Cycle Processes*, ed. IEEE, 2017.
- [4] IEEE standard, *ISO/IEC/IEEE 32675:2022, Information Technology — DevOps: Building Reliable and Secure Systems Including Application Build, Package and Deployment*, ed. IEEE, 2022.
- [5] “ISO/IEC/IEEE 24765:2017 Systems and Software Engineering — Vocabulary,” 2nd ed. 2017
- [6] B. Beyer, C. Jones, J. Petoff, and N.R. Murphy, *Site Reliability Engineering — How Google Runs Production Systems*, O’Reilly Media, 2016.
- [7] *ISO/IEC CD 29110-5-5:2023, Systems and software engineering — Lifecycle profiles for Very Small Entities (VSEs), Part 5-5: Agile/DevOps guidelines*.
- [8] J. Humble and D. Farley. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [9] J. Turnbull, *The Art of Monitoring*. James Turnbull, 2016.